HUMAN EMOTION DETECTION THROUGH TEXT USING ML & NLP

PROJECT REPORT

Submitted to Bharathiar University in partial fulfilment for the requirement of the degree of

BACHELOR OF COMPUTER SCIENCE

(ARTIFICIAL INTELLIGENCE)

Submitted by,

JENSIYA.J (Reg.no:

2128F0003)

Under the Guidance of

Mrs. T.SHANTHI MCA.,M.Phill.,(PhD)

(Assistant Professor of Computer Science)

DEPARTMENT OF ARTIFICIAL INTELLIGENCE

PPG COLLEGE OF ARTS AND SCIENCE

(Affiiliated to Bharathiar University)

SARAVANAMPATTI, COIMBATORE, 641 035

MAR - 2024

CERTIFICATE

BONAFIDE CERTIFICATE

This is the certify that to the project entitled "**HUMAN EMOTION DETECTION THROUGH TEXT USING ML & NLP**"in the bonafide work of **JENSIYA.J (Reg No: 2128F0003)** submitted to Bharathiar University in partial fulfilment of the requirement of the award for degree of Bachelor of Science in **COMPUTER SCIENCE (ARTIFICIAL INTELLIGENCE),** and that this work has not formed the basis of the award of any degree Associate ship or any other title of any other university.

Signature of the Guide

Mrs. T.SHANTHI MCA.,M.Phill(PhD)

Assistant Professor of Computer Science

PPG College of Arts and Science

Signature of the HOD

Mrs. K. YASOTHA MCA.,M.Phill.,(PhD)

Assistant Professor and Head

Department of Computer Science

PPG College of Arts and Science

Viva – Voice examination held on _____

Internal Examiner **External Examiner**

DECLARATION

DECLARATION

I hereby declare that the project entitled "**HUMAN EMOTION DETECTION THROUGH TEXT USING ML & NLP**" by **JENSIYA.J (Reg No: 2128F0003)** in partial fulfilment of the requirement of the award of degree of **B.Sc Computer Science(Artificial Intelligence) at PPG College of Arts and Science** in an authentic record of my carried out under the supervision of **Mrs.T.SHANTHI MCA.,M.Phil.,(PhD) Assistant professor of Computer Science**. The matter presented has not been submitted by me in any other university/institution for the award of degree of **B.Sc. Computer Science.**

PLACE : Coimbatore

Date :

Signature of the candidate

JENSIYA.J

(Reg No: 2128F0003)

ACKNOWLEDGMENT

ACKNOWLEDGEMENT

I wish to convey my heartfelt to our honorable chairman **Dr. L. P. THANGAVELU M.S.,FAIS, FIAGES, FICS, Founder-chairman, PPG Group of Institutions**, who has always encouraged me to give for the best.

At this pleasing moment of having successfully completed out project I wish to convey my sincere thanks and gratitude to our beloved correspondent **Mrs. SHANTHI THANGAVELU** and our trustee **Mr. AKSHAY THANGAVELU** who provide all the facilities to me.

I wish to express my deep gratitude to our beloved **Dr. N. MUTHUMANI MSC.,MPhil.,Ph.D.,NET** and thank her profusely for all the motivation and mortal support.

I heartily thanks **Mrs. K. YASOTHA MCA.,M.Phill.,(PhD)** ,Head of the computer science department, school of computer science as well as assistant professor of school of computer science **Mrs.T.SHANTHI MCA.,M.Phill.,(PhD)** my project guide for providing me all the necessary facilities to complete my project successfully.I express my thanks to other teaching and non-teaching faculty members of computer science department of **PPG College of Arts and Science** for their valuable support and co- operation while working onthis project. Finally, I am grateful to my parents and my friends for showing keen interest and for their encouragement during the course of the project work.

SYNOPSIS

SYNOPSIS

In today's technological world, a majority of users across the world have access to the Internet for communication via text, image, audio and video. People from diverse backgrounds exchange information on current scenarios and project their own views on them over social media. There is a need to understand and recognize the behavior of such large text information on people by analyzing their emotions. The paper focuses on data obtained from one of the most popular social media - Twitter by analyzing live as well as past feeds and getting emotions from them. The twitter data required in English language is converted into a vector of eight emotions and supervised learning techniques such as K-means, Naive Bayes and SVM and NLP is used to determine label identifying one of thebasic emotion family. At the end, a comparative study of the performance of different classifiers is discussed.

TABLE OF CONTENT

TABLE OF CONTENT

1. INTRODUCTION

1.1. ABOUT THE PROJECT

Social media has become an integral part of the people in 21st century. Due to rapid progress in Information & Technology sector, people have access to any kind of information at the click of a button. Moreover, with the invent of smart-phones and 4G networks, even people from the remote areas are getting connected to Tier 1 and Tier 2 cities. With the growing population in countries like India, it has led to tremendous growth in the number of people using social networks. Social networks like Facebook, WhatsApp, Twitter, etc have eliminated the gap between the lives of people.

One of the reasons to use these social networks to know the current happenings around them and to express their views and suggestions in the form of likes, share, tweets, polls, email, etc. This has created a new category of people called netizens. Communication via social media is done in the form of text, image audio and video which contains information and consumes space, memory and Internet bandwidth. All these activities done on social media has resulted into vast amount of information being generated daily. Social media analysis has become a interesting field of research to understand the behavior and thoughts of people in response to social, economic, cultural, educational and all others activities happening around the world. Social media data is in the form of unstructured data since people are from diverse backgrounds with different race, culture, language and standard of livingproject their ideas, views, opinions, expressions and so on the Internet. So, it has become a challengein recent years to extract valuable information from these ever-growing data in the form of posts, emails, blogs, micro-blogs, tweets, reviews, comments, polls and surveys on the Web about an individual, an organization or government in the process of decision-making**.** These opinionated data not only exist on the Web but also within the large organizations like Google, Microsoft, Hewlett-Packard, SAP and SAS to know the opinions of their employees spread across the continents in the form of customer feedback collected from emails and call centers or results from surveys conducted by organizations. This has created strong interest for research in sentiment analysis.

1.1.1 PROBLEM STATEMENT

In today's digital era, vast amounts of textual data are generated daily through various online platforms, including social media, emails, and customer reviews. Understanding human emotions expressed in this text is crucial for numerous applications such as sentiment analysis, customer feedback analysis, mental health monitoring, and personalized recommendation systems. However, manually analyzing this data is time-consuming and prone to biases.

To address this challenge, the problem at hand is to develop an effective machine learning (ML) and natural language processing (NLP) solution for automatically detecting human emotions from textual data. This involves accurately categorizing text into predefined emotion categories such as happiness, sadness, anger, fear, disgust, and surprise.

1.1.2 OBJECTIVES OF THE PROJECT

- **Data Collection**: Gather a diverse and representative dataset containing textual data annotated with corresponding emotion labels.
- **Data Preprocessing**: Clean and preprocess the text data to remove noise,normalize text, and handle issues such as spelling errors, punctuation, andstopwords.
- **Feature Extraction**: Extract relevant features from the preprocessed text data, including word embeddings, n-grams, part-of-speech tags, and sentiment scores.
- **Model Selection**: Explore and evaluate various ML and NLP models suitable foremotion detection tasks, such as Support Vector Machines (SVM), Naive Bayes,Recurrent Neural Networks (RNNs), Convolutional Neural Networks (CNNs), and Transformer-based architectures like BERT or GPT.
- **Model Training**: Train the selected models on the preprocessed text data, tuning hyperparameters and optimizing performance metrics such as accuracy, precision,recall, and F1-score.
- **Model Evaluation**: Evaluate the trained models using appropriate validation techniques such as cross-validation or train-test splits, considering metrics like confusion matrices and ROC curves.
- **Model Deployment**: Deploy the best-performing model into production, integratingit into real-world applications where human emotion detection is required.

1.2 SYSTEM SPECIFICATION

1.2.1 HARDWARE SPECIFICATION

- **Operating System:** Compatible with Windows 10
- **RAM:** 16GB (recommended) or 8GB (minimum requirement)
- **Hard Disk:** 100GB working space (recommended) or 50GB (minimum requirement)
- **Processor :** i3 or i5

1.2.2 SOFTWARE SPECIFICATION

- **Languages:** Python 3.6 or higher
- **IDE:** Python 3.9
- **Technology:** Advanced programming using Python
-

SCOPE OF THE PROJECT

- "Human Emotion Detection through Text using Machine Learning and Natural Language Processing" encompasses several key aspects, including data acquisition, preprocessing, model development, evaluation, deployment, and maintenance.
- Gather a diverse dataset containing textual samples annotated with human emotion labels.
- Clean and preprocess the textual data to remove noise, handle misspellings, punctuation, and other irregularities.
- Extract relevant features from the preprocessed text data, including word embeddings, syntactic features, sentiment scores, and contextual information.
- Explore and implement various machine learning and natural language processing models for emotion detection, including traditional classifiers (e.g., SVM, Naive Bayes) and deep learning architectures (e.g., RNNs, CNNs, transformer-based models like BERT).
- Implement measures to safeguard user data and ensure responsible use of the emotion detection system.

By addressing these components within the defined scope, the project aims to develop an effective and scalable solution for detecting human emotions from textual data, leveraging machine learning and natural language processing techniques to enhance decision-making and user experience in various applications.

SCOPE OF THE PROJECT

- "Human Emotion Detection through Text using Machine Learning and Natural Language Processing" encompasses several key aspects, including data acquisition, preprocessing, model development, evaluation, deployment, and maintenance.
- Gather a diverse dataset containing textual samples annotated with human emotion labels.
- Clean and preprocess the textual data to remove noise, handle misspellings, punctuation, and other irregularities.
- Extract relevant features from the preprocessed text data, including word embeddings, syntactic features, sentiment scores, and contextual information.
- Explore and implement various machine learning and natural language processing models for emotion detection, including traditional classifiers (e.g., SVM, Naive Bayes) and deep learning architectures (e.g., RNNs, CNNs, transformer-based models like BERT).
- Implement measures to safeguard user data and ensure responsible use of the emotion detection system.

By addressing these components within the defined scope, the project aims to develop an effective and scalable solution for detecting human emotions from textual data, leveraging machine learning and natural language processing techniques to enhance decision-making and user experience in various applications.

1.3 SOFTWARE ENVIRONMENT

Programming Languages: Python for its rich ecosystem of machine learning and natural language processing libraries such as NLTK, spaCy, and scikit-learn.

Development Environment: Jupyter Notebooks or integrated development environments (IDEs) like PyCharm for code development and experimentation.

Machine Learning Frameworks: TensorFlow, PyTorch, or Keras for building and training deep learning models.

Text Processing Tools: NLTK and spaCy for text preprocessing tasks such as tokenization, stemming, and part-of-speech tagging.

Data Storage: Databases like SQLite or MongoDB for storing and managing annotated textual data used for model training and evaluation.

Version Control: Git for collaborative development and tracking changes in the project codebase.

Deployment: Docker for containerization and deployment of the developed models as microservices, and cloud platforms such as AWS, Google Cloud, or Microsoft Azure for hosting and scaling the deployed system.

Visualization Tools: Matplotlib, Seaborn, or Plotly for visualizing data distributions, model performance metrics, and emotion detection results.

By leveraging these software components, developers can create a robust and scalable environment for building, training, deploying, and evaluating human emotion detection models from textual data.

2. SYSTEM STUDY

The system study for the project on human emotion detection through text involves a comprehensive analysis of requirements, existing literature, data collection, model development, system design, implementation, and ethical considerations. Initially, stakeholders' requirements are identified to determine the objectives, functionalities, and constraints of the system. Implementation involves developing the system components and deploying them in the chosen environment, ensuring compatibility, scalability, and reliability. Testing and validation are conducted to verify system functionality, performance, and usability.

2.1 EXISTING SYSTEM

In previous Emotion classification can be divided into two different categories: coarse-grained and fine-grained classification. Classifying emotions on a coarse-grained level (positive or negative) canbe accurately perceived from text. Hancock et al. [8] used content analysis Linguistic Inquiry and Word Count (LIWC) to classify emotions as positive or negative. They found that positive emotions are expressed in text by using more exclamation marks and words, while negative emotions are expressed using more affective words. However, this method is limited to positive/negative emotions (happy vs. sad). On the other hand, classifying emotions on a fine grained level (for example, the six Ekman emotions) requires semantic and syntactic analysis of the sentence and can be done using threemethods: (1) Keyword-based detection, (2) Learning-based detection, and (3) Hybrid detection. We discuss each family of methods separately.

DISADVANTAGE

- The existing work will be done without using NLP.
- Have complexities in text classification
- It not only provides a measure of how appropriate a predictor(coefficient size)is, but also its direction of association (positive or negative)

2.1.1 DRAWBACKS OF EXISTING SYSTEM

The absence of Natural Language Processing (NLP) the model may struggle to grasp the contextual and linguistic complexities inherent in human language. Emotions are often expressed through idiomatic phrases, metaphors, and sarcasm, which require sophisticated language understanding capabilities to interpret accurately. By neglecting NLP, the system may overlook these contextual cues, leading to misinterpretations or misclassifications of emotions.

Without NLP techniques limits the ability to extract relevant features from the text that are indicative of emotional content. NLP methods such as sentiment analysis, part-of-speech tagging, and named entity recognition provide valuable insights into the emotional context of the text by identifying sentiment polarity, emotional triggers, and entities associated with specific emotions. Without these features, the emotion detection model may lack the granularity needed to differentiate between subtle variations of emotions or identify the underlying reasons behind emotional expressions.

Moreover, without NLP, the system may struggle with ambiguity and polysemy, where a single word or phrase can have multiple interpretations or emotional connotations depending on the context. NLP techniques help disambiguate such instances by analyzing surrounding words and syntactic structures to infer the intended meaning. Without this contextual understanding, the emotion detection model may produce inconsistent or unreliable results, particularly in cases where the emotional tone is ambiguous or context-dependent.

Furthermore, the absence of NLP may limit the system's adaptability and generalization to different languages, dialects, or cultural expressions of emotion. NLP methods enable cross-lingual and cross-cultural analysis by leveraging language-specific features and linguistic patterns to infer emotions accurately across diverse textual data. Without these capabilities, the emotion detection model may struggle to generalize beyond the training data, leading to biased or inaccurate predictions, especially in multilingual or multicultural contexts.

Text classification poses several complexities due to the inherent nature of language and the diverse ways in which textual data can be expressed. Firstly, one of the significant challenges is the ambiguity and variability of language. Words and phrases can have multiple meanings depending on context, making it difficult for classifiers to accurately assign labels.

Another complexity in text classification is the presence of noise and irrelevant information in the text. Textual data may contain spelling errors, grammatical mistakes, slang, and irrelevant content, which can confuse the classifier and degrade performance. Preprocessing techniques such as text normalization, stopwords removal, and spell checking are necessary to clean the data and reduce noise before classification.

2.2 PROPOSED SYSTEM

Text emotion prediction is a popular application of natural language processing (NLP) that involves determining the sentiment or emotion expressed in a given text. This task can be challenging due to the complexity and variability of human language, as well as the diverse ways in which emotions can be expressed. However, with the advent of machine learning and NLP techniques, it is now possible to build models that can accurately predict emotions from text.One effective approach for text emotion prediction is to use a Random Forest classifier with DictVectorizer. DictVectorizer is a tool that can convert a collection of text documents into a matrix of token occurrences, which can then be used as input to a machine learning algorithm. The Random Forest algorithm is a decision tree-based ensemble method that can handle complex interactions between features, making it well-suited for NLP tasks such as text emotion prediction.

System Requirements Analysis:

It involves identifying and defining the functional and non-functional requirements essential for the development and deployment of the system. Firstly, functional requirements entail specifying the core functionalities the system must perform, such as data acquisition from various sources, preprocessing textual data to remove noise and normalize content, extracting features indicative of emotional content, training and evaluating machine learning models for emotion detection, and providing a user interface for interaction with the system. Additionally, non-functional requirements encompass aspects like performance, scalability, reliability, and security.

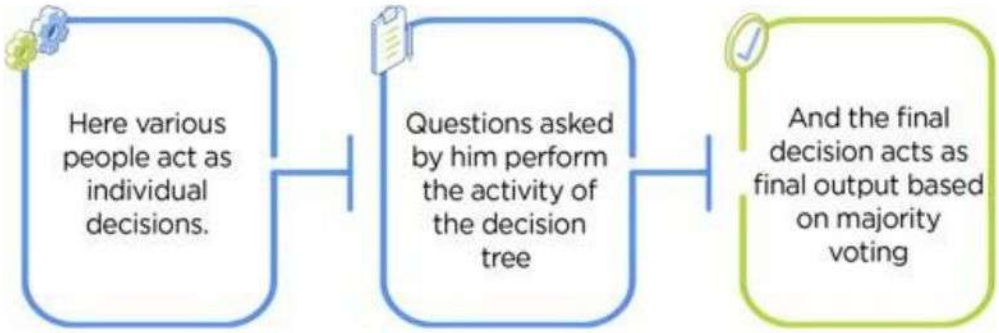
Algorithm Selection:

- **Random Forest Algorithm**

Random Forest Algorithm:-

Random Forest is one of the most popular and commonly used algorithms by Data Scientists. Random forest is a Supervised Machine Learning Algorithm that is used widely in Classification and Regression problems. It builds decision trees on different samples and takes their majority vote for classification and average in case of regression.

One of the most important features of the Random Forest Algorithm is that it can handle the data set containing continuous variables, as in the case of regression, and categorical variables, as in the case of classification. It performs better for classification and regression tasks. In this tutorial, we will understand the working of random forest and implement random forest on a classification task.



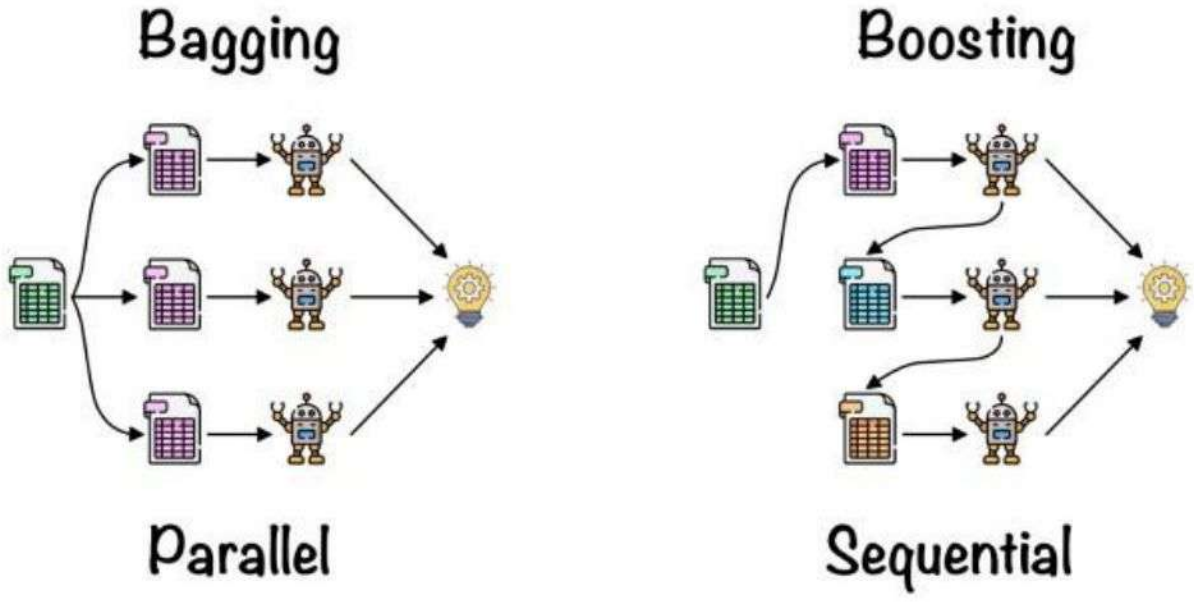
Working of Random Forest Algorithm

Before understanding the working of the random forest algorithm in machine learning, we must look into the ensemble learning technique. Ensemble simplymeans combining multiple models. Thus a collection of models is used to make predictions rather than an individual model.

Ensemble uses two types of methods:

1. Bagging - It creates a different training subset from sample training data with replacement & the final output is based on majority voting. For example, Random Forest.

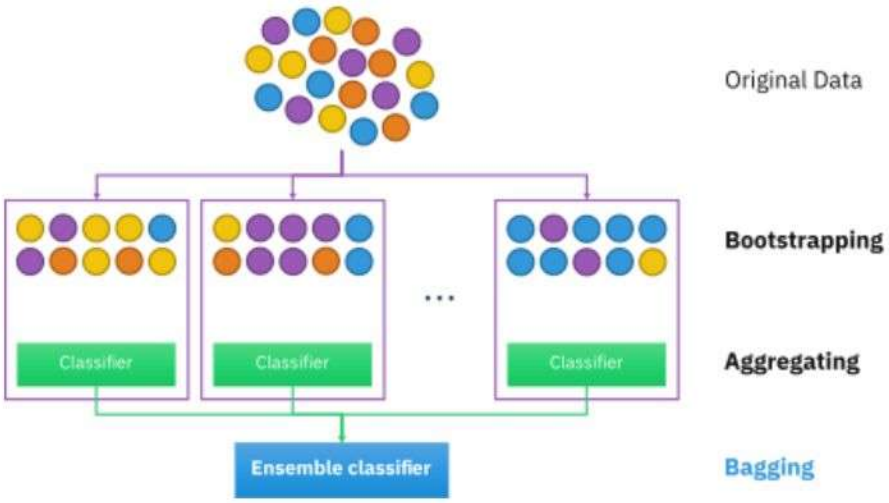
2. Boosting - It combines weak learners into strong learners by creating sequential models such that the final model has the highest accuracy. For example, ADA BOOST, XG BOOST.



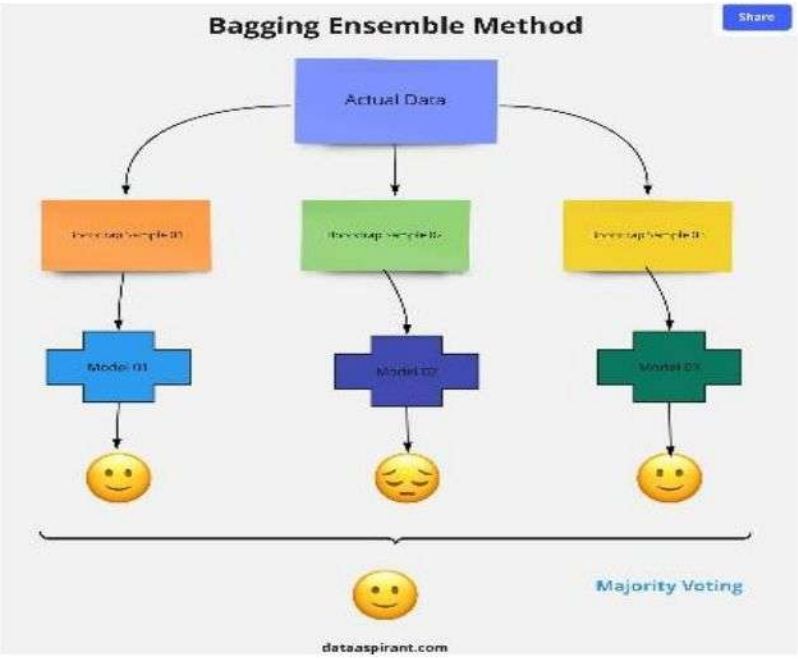
As mentioned earlier, Random forest works on the Bagging principle. Now let's dive in and understand bagging in detail.

Bagging

Bagging, also known as Bootstrap Aggregation, is the ensemble technique used by random forest.Bagging chooses a random sample/random subset from the entire data set. Hence each model is generated from the samples (Bootstrap Samples) provided by the Original Data with replacement known as row sampling. This step of row sampling with replacement is called bootstrap. Now each model is trained independently, which generates results. The final output is based on majority voting after combining the results of all models. This step which involves combining all the results and generating output based on majority voting, is known as aggregation.



Now let's look at an example by breaking it down with the help of the following figure. Here the bootstrap sample is taken from actual data (Bootstrap sample 01, Bootstrap sample 02, and Bootstrap sample 03) with a replacement which means there is a high possibility that each sample won't contain unique data. The model (Model 01, Model 02, and Model 03) obtained from this bootstrap sample is trained independently. Each model generates results as shown. Now the Happy emoji has a majority when compared to the Sad emoji. Thus based on majority voting final output is obtained as Happy emoji.



Boosting

Boosting is one of the techniques that use the concept of ensemble learning. A boosting algorithm combines multiple simple models (also known as weak learners or base estimators) to generate the final output. It is done by building a model by using weak models in series.

There are several boosting algorithms; AdaBoost was the first really successful boosting algorithm that was developed for the purpose of binary classification. AdaBoost is an abbreviation for Adaptive Boosting and is a prevalent boosting technique that combines multiple "weak classifiers" into a single "strong classifier." There are Other Boosting techniques.

Steps Involved in Random Forest Algorithm

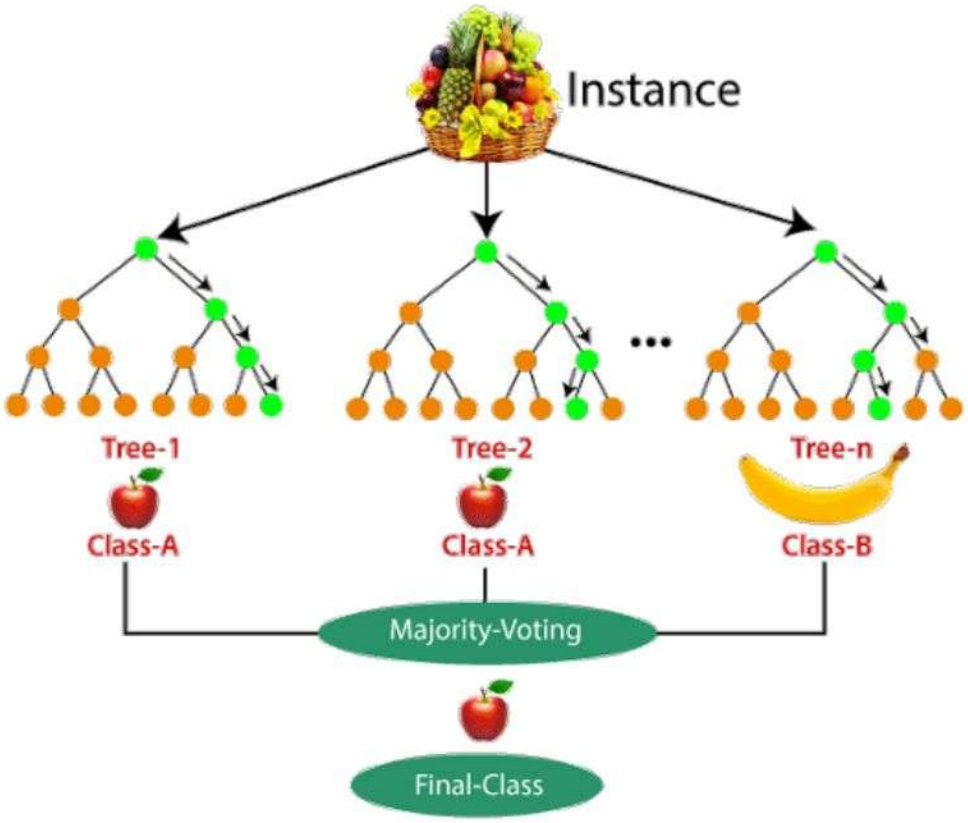
Step 1: In the Random forest model, a subset of data points and a subset of features is selected for constructing each decision tree. Simply put, n random records and m features are taken from the data set having k number of records.

Step 2: Individual decision trees are constructed for each sample.

Step 3: Each decision tree will generate an output.

Step 4: Final output is considered based on Majority Voting or Averaging for Classification and regression, respectively.

For Example: consider the fruit basket as the data as shown in the figure below. Now n number of samples are taken from the fruit basket, and an individual decision tree is constructed for each sample. Each decision tree will generate an output, as shown in the figure. The final output is considered based on majority voting. In the below figure, you can see that the majority decision tree gives output as an apple when compared to a banana, so the final output is taken as an apple.



Important Features of Random Forest

Diversity: Not all attributes/variables/features are considered while making an individual tree; each tree is different. Immune to the curse of dimensionality: Since each tree does not consider all the features, the feature space is reduced.

Parallelization: Each tree is created independently out of different data and attributes. This means we can fully use the CPU to build random forests.

Train-Test split: In a random forest, we don't have to segregate the data for train and test as there will always be 30% of the data which is not seen by the decision tree.

Stability: Stability arises because the result is based on majority voting/ averaging.

Difference Between Decision Tree and Random Forest

Random forest is a collection of decision trees; still, there are a lot of differences in their behavior.

Decision trees	Random Forest
1. Decision trees normally suffer from the problem of overfitting if it's allowed to grow without any control.	1. Random forests are created from subsets of data, and the final output is based on average or majority ranking; hence the problem of overfitting is taken care of.
2. A single decision tree is faster in computation.	2. It is comparatively slower.
3. When a data set with features is taken as input by a decision tree, it will formulate some rules to make predictions.	3. Random forest randomly selects observations, builds a decision tree, and takes the average result. It doesn't use any set of formulas.

Thus random forests are much more successful than decision trees only if the trees are diverse and acceptable.

Important Hyperparameters in Random Forest

Hyperparameters are used in random forests to either enhance the performance and predictive power of models or to make the model faster.

Hyperparameters to Increase the Predictive Power

n_estimators: Number of trees the algorithm builds before averaging the predictions.

max_features: Maximum number of features random forest considers splitting a node.

mini_sample_leaf: Determines the minimum number of leaves required to split an internal node.

criterion: How to split the node in each tree? (Entropy/Gini impurity/Log Loss)

max_leaf_nodes: Maximum leaf nodes in each tree

Hyperparameters to Increase the Speed

n_jobs: it tells the engine how many processors it is allowed to use. If the value is 1, it can use only one processor, but if the value is -1, there is no limit.

random_state:controls randomness of the sample. The model will always produce the same results if it has a definite value of random state and has been given the same hyperparameters and training data.

oob_score: OOB means out of the bag. It is a random forest cross-validation method. In this, one-third of the sample is not used to train the data; instead used to evaluate its performance. These samples are called out-of-bag samples.

Random forest is a great choice if anyone wants to build the model fast and efficiently, as one of the best things about the random forest is it can handle missing values. It is one of the best techniques with high performance, widely used in various industries for its efficiency. It can handle binary, continuous, and categorical data. Overall, random forest is a fast, simple, flexible, and robust model with some limitations.

Data Collection :

- Gather a diverse dataset containing textual samples annotated with human emotion labels. Sources may include social media posts, online forums, customer reviews, and literary works.
- Ensure the dataset covers a wide range of emotions such as happiness, sadness, anger, fear, disgust, and surprise.
- Consider the cultural and linguistic diversity of the dataset to ensure its representativeness across different demographics and language groups.
- Annotate the dataset with accurate emotion labels, either manually or using automated sentiment analysis tools, to create a ground truth for model training and evaluation.

Data Preprocessing :

- Clean the raw textual data to remove noise, including HTML tags, punctuation, special characters, and URLs.
- Normalize the text by converting it to lowercase, removing accents, and standardizing contractions and abbreviations.
- Tokenize the text into words or subword units to create a sequence of tokens for further analysis.
- Handle spelling errors using spell-checking algorithms or dictionaries to improve the accuracy of emotion detection.
- Remove stop words and non-informative words that do not contribute to emotion classification.
- Lemmatize or stem the words to reduce them to their base forms, thereby reducing dimensionality and improving model generalization.

- Handle imbalanced datasets by employing techniques such as oversampling, undersampling, or data augmentation to ensure equal representation of emotion classes.
- Encode emotion labels into numerical values or one-hot encodings suitable for model training.
- Split the preprocessed data into training, validation, and test sets to evaluate the model's performance accurately.

Training (if applicable):

To generate training data set, a total of 100 words from each of the bags is used. These words are first converted into hashtags and then fed into the Tweet Retrieval System (TRS). The TRS keeps on listening to the live tweet stream and captures any tweets with the hashtag that is passed to it. Labelling of training dataset is automated when a tweet is retrieved it is assigned a label which is one of the basic emotion family to which the hashtag word belongs.

Real-time Processing (if applicable):

- **Efficient Text Stream Processing:** Implement a system capable of processing incoming textual data streams in real-time. This requires designing an efficient pipeline that can handle high volumes of text data with low latency. Techniques such as stream processing frameworks (e.g., Apache Kafka, Apache Flink) and microservices architecture can be utilized to ingest, preprocess, and analyze text data in real-time.
- **Scalable Model Inference:** Deploy a lightweight and scalable model for emotion detection that can efficiently process text data in real-time. This involves optimizing the model architecture and inference process to minimize computational overhead while maintaining high accuracy. Techniques like model quantization, pruning, and deploying models on edge devices or cloud-based services can help achieve real-time inference capabilities.

Post-processing and Visualization:

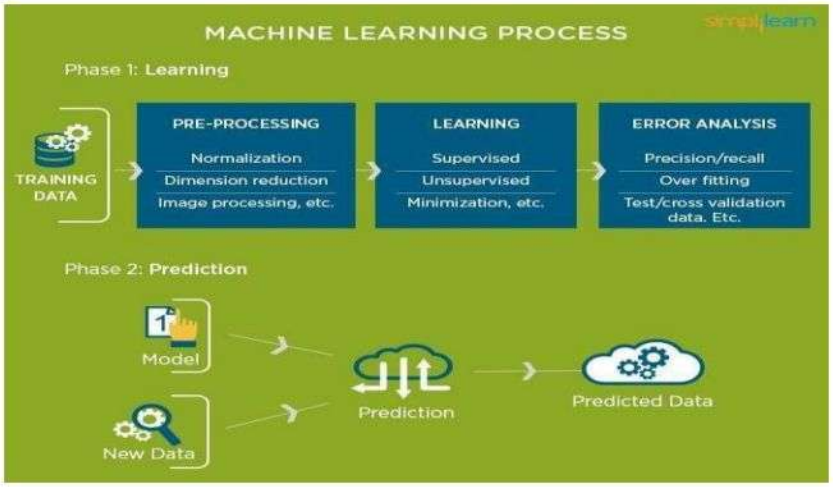
- **Emotion Distribution Analysis:** After emotion detection, perform post-processing to analyze the distribution of detected emotions within the text data. Calculate statistics such as the frequency of each emotion category, proportion of positive/negative emotions, or temporal trends in emotion expression over time. Visualize these distributions using bar charts, pie charts, or line plots to provide insights into the emotional content of the text data.
- **Confidence Level Visualization:** Visualize the confidence levels or probabilities assigned to each detected emotion by the classification model. This can help users understand the model's uncertainty in predicting emotions for different text samples.

2.2.1 SOFTWARE DESCRIPTION

MACHINE LEARNING

To better understand the uses of Machine Learning consider some of the instances where machine learning is applied: the self-driving Google car, cyber fraud detection, online recommendation engines—like friend suggestions on Facebook, Netflix showcasing the movies and shows you might like, and "more items to consider" and "get yourself a little something" on Amazon—are all examples of applied machine learning.

All these examples echo the vital role machine learning has begun to take in today's data-rich world. Machines can aid in filtering useful pieces of information that help in major advancements, and we are already seeing how this technology is being implemented in a wide variety of industries.



USES OF MACHINE LEARNING

Machine learning tasks are classified into several broad categories. In **Supervised Learning**, the algorithm builds a mathematical model of a set of data that contains both the inputs and the desired outputs. For example, if the task were determining whether an image contained a certain object, the training data for a supervised learning algorithm would include images with and without that object (the input), and each image would have a label (the output) designating whether it contained theobject. In special cases, the input may be only partially available, or restricted to special feedback. **Semi-Supervised Learning** algorithms develop mathematical models from incomplete training data, where a portion of the sample inputs are missing the desired output.

Classification algorithms and **Regression** algorithms are types of supervised learning. Classification algorithms are used when the outputs are restricted to a limited set of values. For a classification

algorithm that filters emails, the input would be an incoming email, and the output would be the name of the folder in which to file the email. For an algorithm that identifies spam emails, the output would be the prediction of either "spam" or "not spam", represented by the Boolean values true and false. Regression algorithms are named for their continuous outputs, meaning they may have any value within a range. Examples of a continuous value are the temperature, length, or price of anobject.

SUPERVISED LEARNING

The majority of practical machine learning uses supervised learning. Supervised learning is where you have input variables (x) and an output variable (Y) and you usean algorithm to learn the mapping function from the input to the output **Y = f(X)** . The goal is to approximate the mapping function so well that when you have new input data (x) that you can predict the output variables (Y) for that data.

Techniques of Supervised Machine Learning algorithms

- Linear regression
- Logistic regression,
- Multi-class classification
- Decision Trees
- Support vector machines

Supervised learning requires that the data used to train the algorithm is already labeled with correct answers. For example, a classification algorithm will learn to identify animals after being trained on a dataset of images that are properly labeled with the species of the animal and some identifying characteristics.

Supervised learning problems can be further grouped into **Regression** and **Classification** problems.

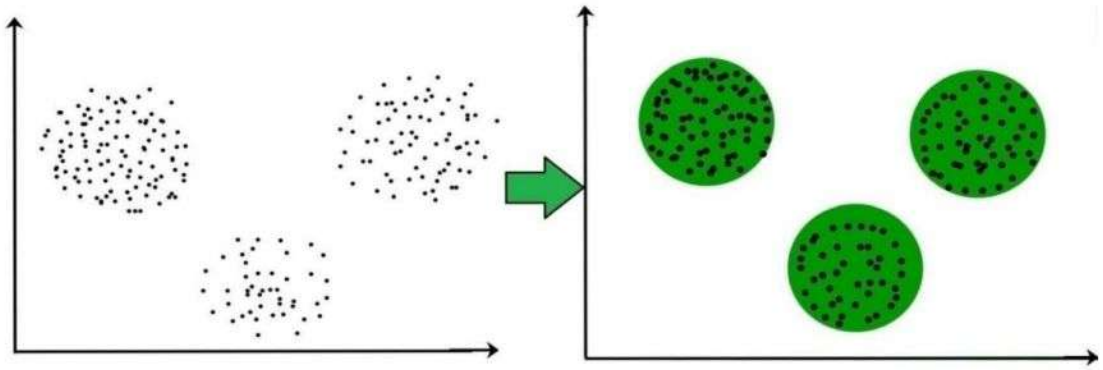
Both problems have as goal the construction of a succinct model that can predict the value of the dependent attribute from the attribute variables. The difference between thetwo tasks is the fact that the dependent attribute is numerical for regression and categorical for classification.

A regression problem is when the output variable is a real or continuous value, such as “salary” or “weight”. Many different models can be used, the simplest is the linear regression. It tries to fit data with the best hyper-plane which goes through the points.

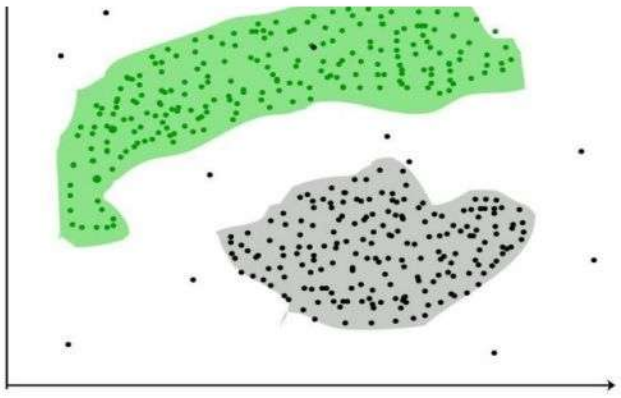
CLUTERING

Clustering is the task of dividing the population or data points into a number of groups such that data points in the same groups are more similar to other data points in the same group and dissimilar to the data points in other groups. It is basically a collection of objects on the basis of similarity and dissimilarity between them.

For ex– The data points in the graph below clustered together can be classified into one single group. We can distinguish the clusters, and we can identify that there are 3 clusters in the below picture.



It is not necessary for clusters to be a spherical. Such as :

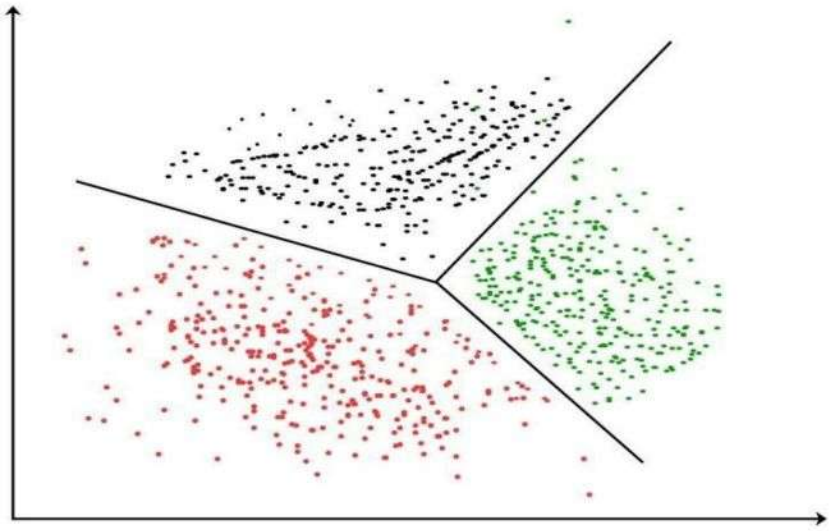


These data points are clustered by using the basic concept that the data point lies within the given constraint from the cluster center. Various distance methods and techniques are used for calculation of the outliers.

CLUSTERING ALGORITHM :

K-means clustering algorithm – It is the simplest unsupervised learning algorithm that solves clustering problem.K-means algorithm partition n observations into k clusters where each observation belongs to the cluster with the nearest mean serving as a prototype of the cluster

.



Applications of Clustering in different fields

1. **Marketing :** It can be used to characterize & discover customer segments formarketing purposes.
2. **Biology :** It can be used for classification among different species of plants andanimals.
3. **Libraries :** It is used in clustering different books on the basis of topics andinformation.
4. **Insurance :** It is used to acknowledge the customers, their policies and identifyingthe frauds.
5. **City Planning :** It is used to make groups of houses and to study their values basedon their geographical locations and other factors present.
6. **Earthquake studies :** By learning the earthquake affected areas we can determinethe dangerous zones.

PRINCIPAL COMPONENT ANALYSIS

The main idea of principal component analysis (PCA) is to reduce the dimensionalityof a data set consisting of many variables correlated with each other, either heavily or lightly, while retaining the variation present in the dataset, up to the maximum extent.

The same is done by transforming the variables to a new set of variables, which are known as the principal components (or simply, the PCs) and are orthogonal, ordered such that the retention of variation present in the original variables decreases as we move down in the order. So, in this way, the 1st principal component retains maximum variation that was present in the original components. The principal components are theeigenvectors of a covariance matrix, and hence they are orthogonal.

Importantly, the dataset on which PCA technique is to be used must be scaled. The results are also sensitive to the relative scaling. As a layman, it is a method of summarizing data. Imagine some wine bottles on a dining table. Each wine is describedby its attributes like colour, strength, age, etc. But redundancy will arise because many of them will measure related properties. So what PCA will do in this case is summarize each wine in the stock with less characteristics.

Intuitively, Principal Component Analysis can supply the user with a lower-dimensional picture, a projection or "shadow" of this object when viewed fromits most informative viewpoint.

Data science is a "concept to unify statistics, data analysis, machine learning and their related methods" in order to "understand and analyze actual phenomena" with data It employs techniques and theories drawn from many fields within the context of mathematics, statistics, information science, and computer science. Data analysis is aprocess of inspecting, cleansing, transforming, and modeling data with the goal of discovering useful information, informing conclusions, and supporting decision-making. Data analysis has multiple facets and approaches, encompassing diverse techniques under a variety of names, while being used in different business, science, and social science domains. In today's business, data analysis is playing a role in making decisions more scientific and helping the business achieve effective operation.

PYTHON:

Python is an open source programming language. Python was made to be easy-to-read and powerful. A Dutch programmer named Guido van Rossum made Python in 1991. He named it after thetelevision show Monty Python's Flying Circus. Many Python examples and tutorials include jokes from the show. Python is an interpreted language. Interpreted languages do not need to be compiled torun. A program called an interpreter runs Python code on almost any kind of computer. This means that a programmer can change the code and quickly see the results. This also means Python is slower than a compiled language like C, because it is not running machine code directly.

Python is a good programming language for beginners. It is a high-level language, which means a programmer can focus on what to do instead of how to do it. Writing programs in Python takes less time than in some other languages.Python drew inspiration from other programming languages like C,C++, Java, Perl, and Lisp.

Python has a very easy-to-read syntax. Some of Python's syntax comes from C, because that is the language that Python was written in. But Python uses whitespace to delimit code: spaces or tabs are used to organize code into groups. This is different from C. In C, there is a semicolon at the end of each line and curly braces ({}) are used to group code. Using whitespace to delimit code makesPython a very easy-to-read language.

Python is used by hundreds of thousands of programmers and is used in many places. Sometimes only Python code is used for a program, but most of the time it is used to do simple jobs while another programming language is used to do more complicated tasks.

Its standard library is made up of many functions that come with Python when it is installed. On the Internet there are many other libraries available that make it possible for the Python language to do more things. These libraries make it a powerful language; it can do many different things.

Some things that Python is often used for are:

- Web development
- Game programming
- Desktop GUIs
- Scientific programming
- Network programming.

Version 1

Python reached version 1.0 in January 1994. The major new features included in this release were the functional programming tools lambda, map, filter and reduce. Van Rossum stated that "Pythonacquired lambda, reduce(), filter() and map(), courtesy of a Lisp hacker who missed them andsubmitted working patches'' The last version released while Van Rossum was at CWI was Python 1.2.In 1995, Van Rossum continued his work on Python at the Corporation for National ResearchInitiatives (CNRI) in Reston, Virginia whence he released several versions.

By version 1.4, Python had acquired several new features. Notable among these are the Modula-3 inspired keyword arguments (which are also similar to Common Lisp's keyword arguments) and built-in support for complex numbers. Also included is a basic form of data hiding by name mangling,though this is easily bypassed.

During Van Rossum's stay at CNRI, he launched the Computer Programming for Everybody (CP4E) initiative, intending to make programming more accessible to more people, with a basic "literacy" in programming languages, similar to the basic English literacy and mathematics skills required by most employers. Python served a central role in this: because of its focus on clean syntax, it was already suitable, and CP4E's goals bore similarities to its predecessor, ABC. The project was funded by DARPA.[13] As of 2007, the CP4E project is inactive, and while Python attempts to be easily learnable and not too arcane in its syntax and semantics, reaching out to non-programmers is not an active concern.

In 2000, the Python core development team moved to BeOpen.com to form the BeOpen Python Labs team. CNRI requested that a version 1.6 be released, summarizing Python's development up to the point at which the development team left CNRI. Consequently, the release schedules for 1.6 and 2.0 had a significant amount of overlap.Python 2.0 was the only release from BeOpen.com. After Python 2.0 was released by BeOpen.com, Guido van Rossum and the other PythonLabs developers joined Digital Creations.

The Python 1.6 release included a new CNRI license that was substantially longer than the CWI license that had been used for earlier releases. The new license included a clause stating that the license was governed by the laws of the State of Virginia. The Free Software Foundation argued that the choice-of-law clause was incompatible with the GNU General Public License. BeOpen, CNRI and the FSF negotiated a change to Python's free software license that would make it GPL-compatible. Python 1.6.1 is essentially the same as Python 1.6, with a few minor bug fixes, and with the new GPL-compatible license.

Version 2

Python 2.0 introduced list comprehensions, a feature borrowed from the functional programming languages SETL and Haskell. Python's syntax for this construct is very similar to Haskell's, apart from Haskell's preference for punctuation characters and Python's preference for alphabetic keywords. Python 2.0 also introduced a garbage collection system capable of collecting reference cycles.[7]

Python 2.1 was close to Python 1.6.1, as well as Python 2.0. Its license was renamed Python Software Foundation License. All code, documentation and specifications added, from the time of Python 2.1's alpha release on, is owned by the Python Software Foundation (PSF), a non-profit organization formed in 2001, modeled after the Apache Software Foundation.[15] The release included a change to the language specification to support nested scopes, like other statically scoped languages.[16] (The feature was turned off by default, and not required, until Python 2.2.)

A major innovation in Python 2.2 was the unification of Python's types (types written in C) and classes (types written in Python) into one hierarchy. This single unification made Python's object model purely and consistently object oriented. Also added were generators which were inspired by Icon.

Python 2.5 was released on September 2006 and introduced the with statement, which encloses a code block within a context manager (for example, acquiring a lock before the block of code is run and releasing the lock afterwards, or opening a file and then closing it), allowing Resource Acquisition Is Initialization (RAII)-like behavior and replacing a common try/finally idiom.

Python 2.6 was released to coincide with Python 3.0, and included some features from that release, as well as a "warnings" mode that highlighted the use of features that were removed in Python 3.0.Similarly, Python 2.7 coincided with and included features from Python 3.1,which was released on June 26, 2009.

Parallel 2.x and 3.x releases then ceased, and Python 2.7 was the last release in the 2.x series. In November 2014, it was announced that Python 2.7 would be supported until 2020, but users were encouraged to move to Python 3 as soon as possible.

Version 3

Python 3.0 (also called "Python 3000" or "Py3K") was released on December 3, 2008 It was designed to rectify fundamental design flaws in the language—the changes required could not be implemented while retaining full backwards compatibility with the 2.x series, which necessitated a new major

version number. The guiding principle of Python 3 was: "reduce feature duplication by removing old ways of doing things".

Python 3.0 was developed with the same philosophy as in prior versions. However, as Python had accumulated new and redundant ways to program the same task, Python 3.0 had an emphasis on removing duplicative constructs and modules, in keeping with "There should be one— and preferably only one —obvious way to do it".

Nonetheless, Python 3.0 remained a multi-paradigm language. Coders still had options among object-orientation, structured programming, functional programming and other paradigms, but within such broad choices, the details were intended to be more obvious in Python 3.0 than they were in Python 2.x.

Python 2.7.0

Note: A bugfix release, 2.7.13, is currently available. Its use is recommended.

Python 2.7.0 was released on July 3rd, 2010.

Python 2.7 is scheduled to be the last major version in the 2.x series before it moves into an extended maintenance period. This release contains many of the features that were first released in Python 3.1. Improvements in this release include:

An ordered dictionary type

New unittest features including test skipping, new assert methods, and test discovery

A much faster io module

Automatic numbering of fields in the str.format() method

Float repr improvements backported from 3.x

Tile support for Tkinter

A backport of the memoryview object from 3.x

Set literals

Set and dictionary comprehensions

Dictionary views

New syntax for nested with statements

Installing Python:

Step 1: Download Python

Visit the official Python website at https://www.python.org/ and go to the "Downloads" section. Choose the desired version for your operating system (Windows, macOS, or Linux).

Step 2: Run the Installer

Windows

- Run the downloaded executable (.exe) file.
- Check "Add Python to PATH" during installation.
- Click "Install Now

Step 3: Verify Installation

- Open a command prompt or terminal and type python --version or python3 --version to check the installed Python version.

Step 4: Virtual Environment (Optional)

Create a virtual environment to isolate projects. Use venv or virtualenv:

- Windows
- Copy code
- python -m venv myenv myenv\Scripts\activate
- macOS/Linux
- bashCopy code
- python3 -m venv myenv source myenv/bin/activate

Step 5: Install Libraries (Optional)

Use pip to install libraries. For example:

pip install library_name

Python is now ready for development, with a rich history of versions catering to evolving programming needs.

3. SYSTEM DESIGN AND DEVELOPMENT

3.1 FILE DESIGN

Designing a file structure for Human Emotion Detection through Text using Machine Learning and Natural Language Processing involves organizing your code, data, and resources in a way that is logical, modular, and easy to maintain. Here's a suggested file structure for an Emotion Detection.

3.2 INPUT DESIGN

Human Emotion Detection involves detecting and recognizing through text within messages or articles etc.NLP is to focuses on enabling computers to understand, interpret, and generate human language. It encompasses tasks such as text classification, sentiment analysis, named entity recognition, machine translation, and more, enabling applications ranging from chatbots and virtual assistants to information retrieval systems and language translation services.

3.3 OUTPUT DESIGN

In the context ofHuman Emotion Detection, the "output design" typically refers to how you visualize or present the results of the Emotion detection algorithm. The most common approach is to the accuracy of the model will depend on the quality and quantity of the data used for training and testing data of an Emotions.

DATABASE DESIGN

For a human emotion detection through text project, is to storing textual data, annotations, and metadata efficiently. Here's a suggested database design:

Text Data Table:

- This table stores the raw textual data along with unique identifiers for each text sample.
- Fields:
    - Text_ID (Primary Key): Unique identifier for each text sample.
    - Text_Content: The raw text data.
    - Timestamp: Timestamp indicating when the text was collected or entered into the system.
    - Source: Source or origin of the text (e.g., social media platform, website).

Emotion Annotations Table:

- This table stores the annotations for each text sample, indicating the detected emotions.
- **Fields:**
    - Annotation_ID (Primary Key): Unique identifier for each annotation.
    - Text_ID (Foreign Key): Reference to the Text_ID in the Text Data Table.
    - Emotion_Category: The detected emotion category (e.g., happiness, sadness, anger).
    - Confidence_Level: Confidence score or probability assigned to the detected emotion.

Metadata Table (Optional):

- This table stores additional metadata associated with each text sample, such as user demographics, location, or context.
- **Fields:**
  - Text_ID (Foreign Key): Reference to the Text_ID in the Text Data Table.
  - User_ID: Identifier for the user who generated or provided the text.
  - Age: Age of the user.
  - Gender: Gender of the user.
  - Location: Location or region associated with the user.
  - Other_Metadata: Additional metadata fields as needed.

With this database design, developers can efficiently store and manage textual data, annotations, and associated metadata for human emotion detection tasks. The design allows for scalability and flexibility, accommodating future enhancements or additional data attributes as needed. Additionally, appropriate indexing and optimization techniques can be applied to improve query performance for data retrieval and analysis.

SYSTEM DEVELOPMENT

To creating comprehensive documentation to guide the development, deployment, and usage of the system. This documentation serves as a reference for developers, stakeholders, and end-users, providing essential information about the project's objectives, architecture, components, and functionality. An overview section provides a high-level description of the project, including its goals, target audience, and scope. This section outlines the problem statement, objectives, and significance of the project in detecting human emotions from textual data

- A system architecture section describes the design and components of the emotion detection system. It outlines the workflow, data flow, and interactions between different modules such as data acquisition, preprocessing, feature extraction, model training, evaluation, and deployment. This section may include diagrams, flowcharts, or system diagrams to illustrate the architecture visually.
- A setup and installation guide provides step-by-step instructions for setting up the development environment, installing dependencies, and configuring the system components. This includes details on installing Python, required libraries (e.g., NLTK, scikit-learn,

TensorFlow), and any additional tools or resources needed for development.

- A usage guide explains how to use the system for emotion detection tasks. It includes instructions for running scripts or applications, providing input data, and interpreting the output results. This section may also cover advanced usage scenarios, customization options, and troubleshooting tips.
- A data documentation section describes the datasets used for training and evaluation, including details on data sources, annotations, preprocessing steps, and data format. This section ensures transparency and reproducibility by providing insights into the dataset characteristics and composition.
- A model documentation section explains the machine learning and NLP models employed in the system for emotion detection. It includes information on model architecture, hyperparameters, training process, evaluation metrics, and model performance. This section may also discuss model limitations, assumptions, and potential areas for improvement then contribution guidelines section outlines how others can contribute to the project, including instructions for submitting bug reports, feature requests, or code contributions. This section encourages collaboration and community involvement in further enhancing the system.

By developing comprehensive documentation covering these aspects, the human emotion detection through text using ML and NLP project ensures clarity, transparency, and accessibility for all stakeholders involved in the development, deployment, and utilization of the system.

Tools

- NUMPY
- PANDAS
- SKLEARN

NUMPY :

- NumPy is a Python package. It stands for 'Numerical Python'.
- It is a library consisting of multidimensional array objects and a collection of routines for processing of array.
- Numeric, the ancestor of NumPy, was developed by Jim Hugunin.
- Another package Numarray was also developed, having some additional functionalities.
- In 2005, Travis Oliphant created NumPy package by incorporating the features of Numarray into Numeric package.
- There are many contributors to this open source project.

Operations using NumPy :

Using NumPy, a developer can perform the following operations

- Mathematical and logical operations on arrays.
- Fourier transforms and routines for shape manipulation.
- Operations related to linear algebra. NumPy has in-built functions for linear algebra and random number generation.

NumPy – A Replacement for MatLab NumPy is often used along with packages like SciPy (Scientific Python) and Mat−plotlib (plotting library). This combination is widely used as areplacement for MatLab, a popular platform for technical computing. However, Python alternative to MatLab is now seen as a more modern and complete programming language.

It is open source, which is an added advantage of NumPy.

INSTALLATION

pip install "numpy”

PANDAS

- Pandas is an open-source, BSD-licensed Python library providing high-performance,easy-to-use data structures and data analysis tools for the Python programming language.

- Python with Pandas is used in a wide range of fields including academic and commercial domains including finance, economics, Statistics, analytics, etc. In this tutorial, we will learn the various features of Python Pandas and how to use them in practice.

Pandas deals with the following three data structures

- Series
- DataFrame
- Panel

These data structures are built on top of Numpy array, which means they are fast.

INSTALLATION

Pip install pandas

SKLEARN

- Scikit-learn is a machine learning library for Python.
- It features several regression,classification and clustering algorithms including SVMs, gradient boosting, k-means, random forests and DBSCAN. It is designed to work with Python Numpy and SciPy .
- The scikit-learn project kicked off as a Google Summer of Code (also known as GSoC) project by David Cournapeau as scikits.learn. It gets its name from "Scikit", a separate third-party extension to SciPy.

Python Scikit-learn

Scikit is written in Python (most of it) and some of its core algorithms are written in Cython for even better performance. Scikit-learn is used to build models and it is not recommended to use it for reading, manipulating and summarizing data as there are better frameworks available for the purpose.It is open source and released under BSD license.

Installation of Scikit Learn

Scikit assumes you have a running Python 2.7 or above platform with NumPY

(1.8.2 and above) and SciPY (0.13.3 and above) packages on your device. Once we

have these packages installed we can proceed with the installation.

pip install scikit-learn

Flask:

What is Flask?

- Flask is an API of Python that allows us to build up web-applications. It was developed by Armin Ronacher
- Flask's framework is more explicit than Django's framework and is also easier to learn because it has less base code to implement a simple web-Application.
- A Web-Application Framework or Web Framework is the collection of modules and libraries that helps the developer to write applications without writing the low-level codes such as protocols, thread management, etc.
- Flask is based on WSGI(Web Server Gateway Interface) toolkit and Jinja2 template engine.

Routing:

- Nowadays, the web frameworks provide routing technique so that user can remember the URLs.
- It is useful to access the web page directly without navigating from the Home page. It is done through the following route() decorator, to bind the URL to a function.
- Building URL in FLask:
- Dynamic Building of the URL for a specific function is done using url_for() function.
- The function accepts the name of the function as first argument, and one or more keyword arguments.

MODULES

- Data collection
- Data Pre-Processing
- Training data and Test data
- Model Creation
- Model Prediction

3.4 DESCRIPTION OF MODULE

□ Data collection

To generate training data set, a total of 100 words from each of the bags is used. These words are first converted into hashtags and then fed into the Tweet Retrieval System (TRS). The TRS keeps on listening to the live tweet stream and captures any tweets with the hashtag that is passed to it. Labelling of training dataset is automated when a tweet is retrieved it is assigned a label which is one of the basic emotion family to which the hashtag word belongs

□ Data Pre-Processing

Pre-processing refers to the transformations applied to our data before providing the data to the algorithm. Data Preprocessing technique is used to convert the raw data into an understandable data set. In other words, whenever the information is gathered from various sources it is collected in raw format that isn't possible for the analysis.

□ Training data and Test data

- For choosing a model we split our dataset into train and test
- Here data's are split into 3:1 ratio that means
- Training data having 70 percent and testing data having 30 percent
- In this split process preforming based on train_test_split model
- After splitting we get xtrain xtest and ytrain ytest

□ Model Creation

- Contextualise machine learning in your organisation.
- Explore the data and choose the type of algorithm.
- Prepare and clean the dataset.
- Split the prepared dataset and perform cross validation.
- Perform machine learning optimisation.
- Deploy the model.

□ Model Prediction

Predictive modeling is a statistical technique using machine learning and data mining topredict and forecast likely future outcomes with the aid of historical and existing data. It works by analyzing current and historical data and projecting what it learns on a model generated to forecast likely outcomes.

In this project our final prediction is to predict the emotion based on the given text.

4. TESTING AND IMPLEMENTATION

Human emotion detection through text using Machine Learning (ML) and Natural Language Processing (NLP) involves leveraging advanced computational techniques to analyze textual data and infer the underlying emotions expressed within the text.

MODULE TESTING

- Test the model training module to ensure that machine learning models are trained correctly on the feature-engineered data.
- Verify that the selected ML/NLP models converge during training and achieve satisfactory performance metrics.
- Test different hyperparameter settings and optimization algorithms to assess their impact on model training.

.

UNIT TESTING

- Test each preprocessing step individually to ensure it handles input text correctly. For example, test tokenization, stopwords removal, and stemming/lemmatization functions to verify that they produce the desired output.
- Use sample text inputs with known outputs to validate the correctness of each preprocessing step.

SYSTEM TESTING

ML models are trained on annotated datasets to classify text into emotion categories, while NLP techniques enable understanding of linguistic nuances, sentiment analysis, and context modeling to enhance emotion detection accuracy. By leveraging ML and NLP synergistically, this approach facilitates the development of robust systems capable of accurately identifying and interpreting human emotions in textual data, enabling applications such as sentiment analysis, affective computing, and personalized user experiences.

OUTPUT TESTING

the output assessment involves evaluating the model's ability to accurately classify emotions in unseen text samples. This includes measuring metrics such as accuracy, precision, recall,

and F1-score to assess the model's performance across different emotion categories. Additionally, qualitative analysis may be conducted to examine the model's predictions in context, identifying any instances of misclassification or ambiguity. Through rigorous testing and validation, the effectiveness and reliability of the emotion detection system can be verified, ensuring its suitability for real-world applications and user interactions.

IMPLEMENTATION

- Preprocessing techniques such as tokenization, stemming, and stopword removal are applied to clean and normalize textual data.
- Next, feature engineering methods extract relevant linguistic features, sentiment scores, and contextual information from the text.
- ML models, including deep learning architectures like recurrent neural networks (RNNs) or transformer-based models such as BERT, are trained on annotated datasets to classify text into emotion categories.
- NLP techniques enhance model interpretability and performance, enabling accurate detection of subtle emotional nuances.
- Finally, the trained model is deployed into production environments, leveraging scalable infrastructure and APIs for real-time emotion detection applications.
- Through this implementation process, human emotion detection systems can effectively analyze and interpret textual data, enriching user experiences across various domains.

Setup Environment:

Install Packages: Install the libraries using pip for Python or your system's package manager.

Import Libraries: Import the necessary libraries, including Pandas,NumPy, SKlearn in your Python script.

Load Pre-trained Model:

Download a pre-trained Emotion detection model from the other sources. Popular choices include Haar cascades for simpler emotions of machine learning-based models like Random Forest, Natural Language Processing for more complex Emotions Detections.

Input Data:

This kind of Emotion Sentences

- When I was involved in a traffic accident
- I have a fear of dog
- My dog died yesterday
- I don't love you anymore..!

Preprocess Input Data: Preprocess the input data as needed, such as converting into binary using functions to the accurate result for Human Emotions detection model.

Post-processing:

Process Detection Results: Parse the detection results and extract sentences,accuracy scores, and class labels as needed.

Visualize Results:

Through Emojis: emoji_dict = {"joy":"  ", "fear":"  ", "anger":"  ",

"sadness":"  ", "disgust":"  ", "shame":"  ", "guilt":"  "}

Display Results: Show the processed Emojis for apt emotion sentences.

Output Results:

Save Output: Save the processed sentences with emojis overlaid to disk for further analysis or visualization.

Cleanup:

Release Resources: Release any resources and perform cleanup operations to free up memory.

Integration and Deployment:

Integrate the Text emotion detection implementation into your larger system or application as needed.

Deploy the system, ensuring compatibility with target environments and platforms.

Testing and Evaluation:

Test the Text emotion detection implementation using various input data and scenarios to evaluate its performance, accuracy, and robustness.

Iterate on the implementation based on testing results to improve its effectiveness and address any issues.

5. CONCLUSION

The most important part is getting good quality of data. Often tweets retrieved consists of only URLs and hence pre-processing data still remains one of the most crucial steps which needs to be improved. Secondly, only explicit emotions are explored in the tweet so detecting implicit emotions in objective sentences can in included in future work. At present, each sentence is treated as individual unit. Further the association between the sentences can be included. The proposed system works at the syntactic level only, semantic level parsing can be implemented further. Finally, the Bag of words needs to be more exhaustive and hence more words describing emotions can be added to these bags In conclusion, using a Random Forest classifier with DictVectorizer for text emotion prediction can be a highly effective approach. DictVectorizer is useful for converting text data into a format that can be used by the classifier, while the Random Forest algorithm can handle complex interactions between features and can provide robust predictions. Additionally, Random Forest models are less prone to overfitting compared to other models, making them a good choice for this task.Overall, the accuracy of the model will depend on the quality and quantity of the data used for training and testing. It is also important to consider other factors such as feature selection and preprocessing techniques, as well as the specific requirements of the problem at hand. Nonetheless, the combination of DictVectorizer and Random Forest can be a powerful tool for text emotion prediction, with potential applications in sentiment analysis, customer feedback analysis, and social media monitoring, among others.

6. BIBLIOGRAPHY & REFERENCES

BOOK REFERENCE

1] Bing Liu (2009) Sentiment Analysis and Subjectivity, Handbook

- YasminaDouiji and HazarMousanifI-CARE - Intelligent Context Aware system for Recognizing Emotions from text
- Agarwal A 2012 Unsupervised Emotion Detection from Text using Semantic and Syntactic Relations IEEE/WIC/ACM International Conference on Web Intelligence and ntelligent Agent Technology pp 346-353
- Haxby J V, Hoffman and Gobbini M I 2002 Human neural system for face recognition and social communicationBiol. Pyschiatry51pp 59-67
- Nilesh 2014 Approaches of Emotion Detection from Text International Journal of Computer Science and Information Technology Research2pp 123-128\
- Ekman P 2015 An Argument for Basic Emotion International Journal of Cognition andEmotion 6

- Chatterjee A, Gupta U, Chinnakotla MK, Srikanth R, Galley M, Agrawal P. Understanding emotions in text using deep learning and big data. Comput Hum Behav. 2019;93:309–17.

- Rodríguez AOR, Riaño MA, García PAG, Marín CEM, Crespo RG, Wu X. Emotional characterization of children through a learning environment using learning analytics and AR-Sandbox. J Ambient Intell Human Comput. 2020;11:1–15.

- Hossain MS, Muhammad G. Emotion recognition using deep learning approach from audio–visual emotional big data. Inf Fusion. 2019;49:69–78.

- Zhang H, Jolfaei A, Alazab M. A face emotion recognition method using convolutional neural network and image edge computing. IEEE Access. 2019;7:159081–9.

- Kanjo E, Younis EM, Ang CS. Deep learning analysis of mobile physiological, environmental and location sensor data for emotion detection. Inf Fusion. 2019;49:46–56.

- Chen T, Ju S, Yuan X, Elhoseny M, Ren F, Fan M, et al. Emotion recognition using empirical mode decomposition and approximation entropy. Comput Electr Eng. 2018;72:383–92.

- Shrivastava K, Kumar S, Jain DK. An effective approach for emotion detection in multimedia text data using sequence based convolutional neural network. Multimed Tools Appl. 2019;78(20):29607–39.

- Asghar MZ, Subhan F, Ahmad H, Khan WZ, Hakak S, Gadekallu TR, et al. Senti-eSystem: a sentiment-based eSystem-using hybridized fuzzy and deep neural network for measuring customer satisfaction. Software Pract Exper. 2021;51(3):571–94.

- Hasan M, Rundensteiner E, Agu E. Automatic emotion detection in text streams by analyzing twitter data. Int J Data Sci Analytics. 2019;7(1):35–51.

- Alazab R. 0038 children below 5 years of employed mothers are less exposed to acute poisoning in Alexandria, Egypt. Occup Environ Med. 2014;71(Suppl 1):A63.

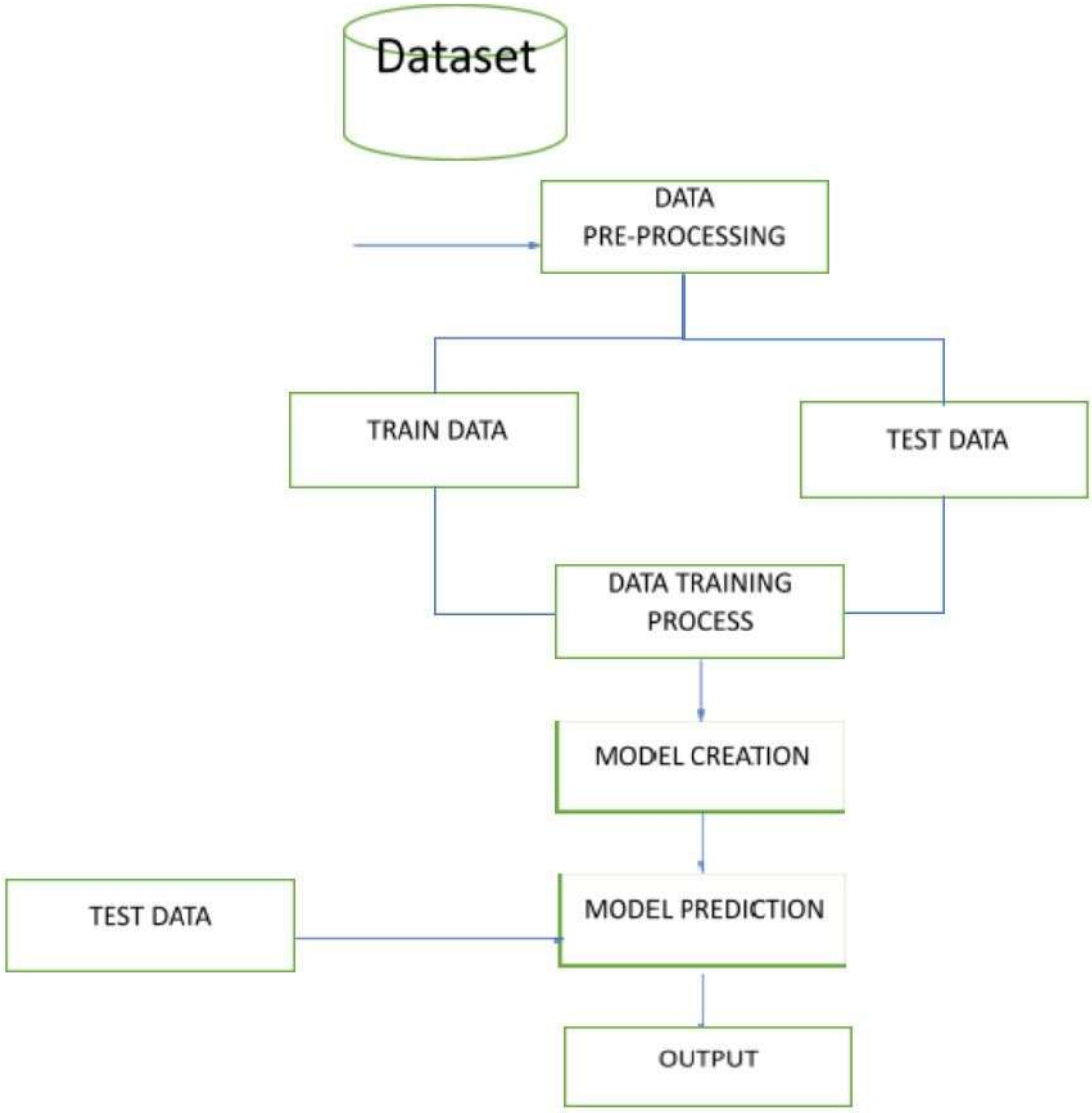
- Dash S, Luhach AK, Chilamkurti N, Baek S, Nam Y. A Neuro-fuzzy approach for user

- behaviour classification and prediction. J Cloud Comput. 2019;8(1):17.

- Awad A, Obayan A, Salhab S, Roufayel R, Kadry S. Effect of smoking on appetite, concentration and stress level. Glob J Health Sci. 2020;12(1):139–9.

7. APPENDICES

A. DATA FLOW DIAGRAM



PREDICTION

Robust predictions: Random Forest models are less prone to overfitting compared to other models, making them a good choice for text emotion prediction. This is because each tree in the forest is trained on a random subset of the data, which reduces the variance in the predictions.

FEATURE DETECTION

Random Forest can provide information on the importance of each feature in the model. This can be useful for understanding which words or features are most important in predicting emotions, and can help guide further analysis.

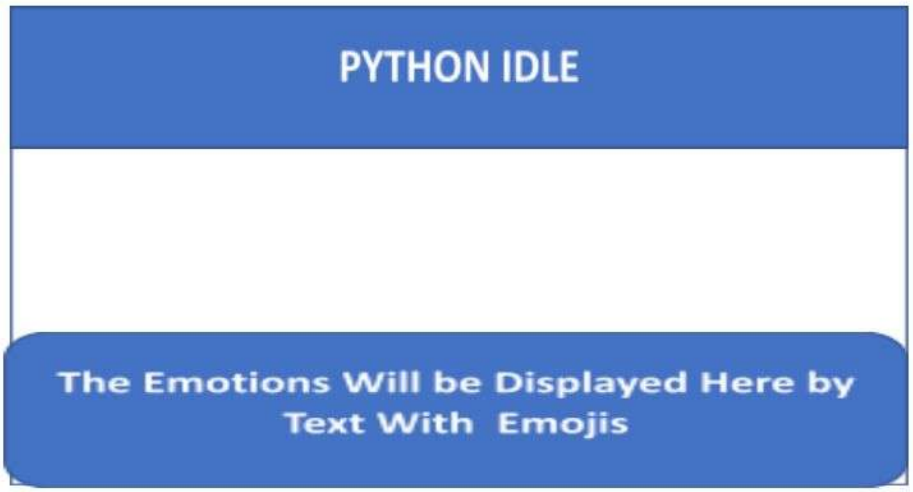
TEXT EMOTION DETECTION

Handles high-dimensional data: NLP tasks often involve high-dimensional data, where each document may contain hundreds or thousands of features. Random Forest can handle this type of data efficiently, making it a scalable and effective method for text emotion prediction.

B. SCREENSHOT

Input Design

Output Design



C. SOURCE CODE

```
import re
import nltk
nltk.download('punkt')#divides a text into a list of sentences
nltk.download('wordnet')
from collections import Counter
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.svm import SVC
from sklearn.svm import LinearSVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
import warnings
warnings.filterwarnings("ignore")
def read_data(file):
    data = []
    with open(file, 'r')as f:
        for line in f:
            line = line.strip()
            label = ' '.join(line[1:line.find("]")].strip().split())
            text = line[line.find("]")+1:].strip()
            data.append([label, text])
    return data

file = 'text.txt'
data = read_data(file)
print("Number of instances: {}".format(len(data)))

def ngram(token, n):
    output = []
    for i in range(n-1, len(token)):
        ngram = ' '.join(token[i-n+1:i+1])
        output.append(ngram)
    return output

def create_feature(text, nrange=(1, 1)):
    text_features = []
    text = text.lower()
    text_alphanum = re.sub('[^a-z0-9#]', ' ', text)
    for n in range(nrange[0], nrange[1]+1):
        text_features += ngram(text_alphanum.split(), n)
    text_punc = re.sub('[a-z0-9]', ' ', text)
    text_features += ngram(text_punc.split(), 1)
    return Counter(text_features)
def convert_label(item, name):
    items = list(map(float, item.split()))
    label = ""
    for idx in range(len(items)):
        if items[idx] == 1:
            label += name[idx] + " "

    return label.strip()
```

```
emotions = ["joy", 'fear', "anger", "sadness", "disgust", "shame", "guilt"]

X_all = []
y_all = []
for label, text in data:
    y_all.append(convert_label(label, emotions))
    X_all.append(create_feature(text, nrange=(1, 4)))

X_train, X_test, y_train, y_test = train_test_split(X_all, y_all, test_size = 0.2, random_state = 123)

def train_test(clf, X_train, X_test, y_train, y_test):
    clf.fit(X_train, y_train)
    train_acc = accuracy_score(y_train, clf.predict(X_train))
    test_acc = accuracy_score(y_test, clf.predict(X_test))
    return train_acc, test_acc

from sklearn.feature_extraction import DictVectorizer
vectorizer = DictVectorizer(sparse = True)
X_train = vectorizer.fit_transform(X_train)
X_test = vectorizer.transform(X_test)

svc = SVC()
lsvc = LinearSVC(random_state=123)
rforest = RandomForestClassifier(random_state=123)
dtree = DecisionTreeClassifier()

clifs = [svc, lsvc, rforest, dtree]

# train and test them
print("| {:25} | {} | {} |".format("Classifier", "Training Accuracy", "Test Accuracy"))
print("| {} | {} | {} |".format("-"*25, "-"*17, "-"*13))
for clf in clifs:
    clf_name = clf.__class__.__name___
    train_acc, test_acc = train_test(clf, X_train, X_test, y_train, y_test)

    print("| {:25} | {:17.7f} | {:13.7f} |".format(clf_name, train_acc, test_acc))

emoji_dict = {"joy":" ", "fear":" ", "anger":" ", "sadness":" ",
"disgust":" ", "shame":" ", "guilt":" "}

t1 = "When I was involved in a traffic accident."
t2 = "I have a fear of dogs"
t3 = "My dog died yesterday"
t4 = "I don't love you anymore..!"

texts = [t1, t2, t3, t4]
for text in texts:
    features = create_feature(text, nrange=(1, 4))
    features = vectorizer.transform(features)
    prediction = clf.predict(features)[0]
    print( text,emoji_dict[prediction])
```

OUTPUT SCREENSHOT

```
it  Shell  Debug  Options  Window  Help
 3.9.6 (tags/v3.9.6:db3ff76, Jun 28 2021, 15:26:21) [MSC v.1929 64 bit (AMD64)] on win32
help", "copyright", "credits" or "license()" for more information.

ART: D:\HICAS\HICAS code\text emotion dtection using machine learning and NLP\emotion.py
 of instances: 7480
sifier               | Training Accuracy | Test Accuracy |
-------------------- | ----------------- | ------------- |
                     |         0.9067513 |     0.4512032 |
arSVC                |         0.9988302 |     0.5768717 |
omForestClassifier   |         0.9988302 |     0.5541444 |
sionTreeClassifier   |         0.9988302 |     0.4545455 |
    (1. 0. 0. 0. 0. 0. 0.)  1084
    (0. 0. 1. 0. 0. 0. 0.)  1080
s   (0. 0. 0. 1. 0. 0. 0.)  1079
    (0. 1. 0. 0. 0. 0. 0.)  1078
t   (0. 0. 0. 0. 1. 0. 0.)  1057
    (0. 0. 0. 0. 0. 0. 1.)  1057
    (0. 0. 0. 0. 0. 1. 0.)  1045
ooks so impressive 😢
 a fear of dogs 😱
 died yesterday 😐
t love you anymore..! 😂
```